

GTU ROBOTICS CLUB

Deployment Of Your Trained Model In YOLOv8 In Local Environment

YOLOv8 Deployment in VS Code - Comprehensive Guide

Introduction

This guide provides a step-by-step walkthrough on deploying a custom-trained YOLOv8 model in VS Code. It covers environment setup, loading the model, running inference, and optimizing performance. From this guide you also how to use your Train model for Real Time data with **Intel RealSense Viewer**.

First of all, clarify that this guide is intended for use after the model has been **created** and **trained**. This guide specifically focuses on how to deploy the **trained model** in a local environment (PyCharm, VS Code).

Prerequisites

Before proceeding, We will download everything which is given below:

- Python (Recommended: Python 3.8 or later for Image Processing)
- VS Code installed with Python extension
- Ultralytics YOLOv8 installed
- Trained YOLOv8 model (weights file in .pt format)
- CUDA (for GPU acceleration) (Optional but recommended)

Step 1: Setting Up the Environment

1. Install Python(Recommended: Python 3.8 or later):

Download and install the Python from the official website:

[Download Python 3.10.0](<https://www.python.org/downloads/>)

2. IDE Setup:

****VS Code****

- Install Visual Studio Code:
[Download VS Code](<https://code.visualstudio.com/>)
- Install the Python extension from the Extensions Marketplace within VS Code.

3. Test your first program:

After downloading and installing **VS Code** and **Python**, create a dedicated folder where you will store all your **.py** files. This will help keep your projects organized.

Writing Your First Python Program in VS Code:

1. **Open VS Code** and navigate to the folder you created.
2. **Create a new Python file** (e.g., **hello.py**).
3. **Write the following simple program:**

```
print("Hello, World!")
```
4. **Save the file** and open the integrated terminal in VS Code.
5. If everything is set up correctly, you should see the output:
Hello, World!

Yes! If you successfully ran **hello.py** in VS Code, that means:

- ✓ Python is installed correctly.
- ✓ VS Code is set up with the right interpreter.
- ✓ Your environment is working as expected.

Step 2: Create a Virtual Environment For YOLOv8

Don't skip anything in this part:

To set up a **YOLOv8s** virtual environment , follow these steps:

1. Navigate to Your Project Folder

Since after created a new folder for running **.py** files, open a terminal and go to that folder:

Command: `cd /path/to/your/folder`

Note: (Replace `/path/to/your/folder` with the actual path.)

After above step you see that you reached at your folder:

2. Create a Virtual Environment:

Command: `python3 -m venv yolov8_env`

This creates a virtual environment named `yolov8_env`.

3. Activate the Virtual Environment

Command: `source yolov8_env/bin/activate`
(# On Windows use: `yolov8_env\Scripts\activate`)

Once activated, you'll see `(yolov8_env)` at the beginning of your terminal prompt.
You can also see in your **Project Folder**.

4. Testing YOLOv8 in Your Virtual Environment (Detailed Explanation)

After installing YOLOv8, you should test if everything is working correctly. This step ensures that the model is properly installed and can run inference (object detection) on an image.

Run this command inside your activated virtual environment:

Command :
`yolo task=detect mode=predict model=yolov8s.pt`
`source='https://ultralytics.com/images/bus.jpg'`

`yolo`: Runs the YOLOv8 command-line tool.

`task=detect`: Specifies that we are performing **object detection**.

`mode=predict`: Tells YOLOv8 to **predict (detect objects)** on an image.

`model=yolov8s.pt`: Uses the **YOLOv8s (small) pre-trained model**.

`source='https://ultralytics.com/images/bus.jpg'`: Downloads an image of a bus from the Ultralytics website and runs object detection on it.

What Will Happen?

- **The model will be downloaded** (if not already present) and stored in `~/.cache/ultralytics/` directory.
- **YOLOv8 will process the image** and detect objects like **people, buses, and cars**.
- **The detected image will be saved in the `runs/detect/predict/` folder** inside your project directory.

How to View the Result?

Command : `xdg-open runs/detect/predict/bus.jpg`

Running on a Local Image

If you have an image stored locally (e.g., `test.jpg` in your project folder), you can run:

Command : `yolo task=detect mode=predict model=yolov8s.pt source=test.jpg`

(Replace `test.jpg` with your actual image file.)

Running on a Video

To detect objects in a video (e.g., `video.mp4` in your project folder), use:

Command : `yolo task=detect mode=predict model=yolov8s.pt source=video.mp4`

The output video will be saved in `runs/detect/predict/`

If you see objects detected in the images/videos, congratulations!
🎉 YOLOv8 is working perfectly. 🚀

Step 3: Install Required Libraries

Open the terminal and run all this command step by step:

1. OpenCV (`opencv-python`):

Command : `pip install opencv-python`

2. Numpy:

Command : `pip install numpy`

3. RealSense SDK ([pyrealsense2](#)):

Command : pip install pyrealsense2

4. Ultralytics YOLO ([ultralytics](#)):

Command : pip install ultralytics

5. Matplotlib

Command : pip install matplotlib

6. Pytorch

Command : pip install torch

Step 4: Intel RealSense Viewer Environment Setup

Step A: Download the Required SDK driver(Always prefer latest version):

LINK : <https://github.com/IntelRealSense/librealsense/releases/tag/v2.56.3>

go to Asset Section and Download:

LINK :

<https://github.com/IntelRealSense/librealsense/releases/download/v2.56.3/Intel.RealSense.SDK-WIN10-2.56.3.7838.beta.exe>

Step B: Testing the camera with your Intel RealSense Viewer After above link:

Connect the RealSense camera with your USB port.

Check the SDK Driver is properly working or not.

Note:Please watch the videos on YouTube for how to use the camera with Intel RealSense Viewer and configure its settings.

Step 5: Testing Everything Step by Step in Local Environment

We have done all requirement setup in our laptop now we see in this step.

In this step we will do :

- 1) **YOLOv8s** Testing on image:(Pretrained model)

For the testing the on **YOLOv8s** on image in VS Code:

```
# Import required libraries
from ultralytics import YOLO # YOLO class for loading the model
import matplotlib.pyplot as plt # Library for displaying the image
import cv2 # OpenCV for image processing

# Load the YOLOv8 model
model = YOLO("yolov8s.pt") # Load the pre-trained YOLOv8-small model

# Load the image
image_path = "C:/Users/HP/Pictures/img_9.jpeg.jpg" # Specify the path
to the input image
image = cv2.imread(image_path) # Read the image using OpenCV

# Check if the image is loaded successfully
if image is None:
    raise FileNotFoundError(f"Error: The image at {image_path} could
not be loaded. Check the file path.")

# Run inference on the image
results = model(image) # Perform object detection using YOLOv8

# Process and display results
for result in results:
    annotated_img = result.plot() # Generate an image with detected
objects

# Convert BGR to RGB for correct color representation in Matplotlib
plt.imshow(annotated_img[..., ::-1]) # OpenCV loads images in BGR
format; converting to RGB
plt.axis("off") # Hide axes for better visualization
plt.show() # Display the annotated image
```

2) **RealSense** Testing on VS code:

For the testing the on **RealSense** in VS Code:

```

import pyrealsense2 as rs
import numpy as np
import cv2

# Initialize RealSense pipeline
pipeline = rs.pipeline()
config = rs.config()

# Start the pipeline
config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)
config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)

pipeline.start(config)

try:
    while True:
        # Wait for frames
        frames = pipeline.wait_for_frames()

        # Get depth and color frames
        depth_frame = frames.get_depth_frame()
        color_frame = frames.get_color_frame()

        if not depth_frame or not color_frame:
            continue

        # Convert frames to numpy arrays
        depth_image = np.asanyarray(depth_frame.get_data())
        color_image = np.asanyarray(color_frame.get_data())

        # Show images
        cv2.imshow('RealSense Color Stream', color_image)
        cv2.imshow('RealSense Depth Stream', depth_image)

        # Break loop on 'q' key press
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

finally:
    # Stop pipeline and close windows
    pipeline.stop()
    cv2.destroyAllWindows()

```

3) RealSense with YOLOv8s Testing on VS code:

```
import pyrealsense2 as rs # Intel RealSense SDK for accessing camera
streams
import numpy as np # NumPy for numerical operations
from ultralytics import YOLO # YOLO for object detection
import matplotlib.pyplot as plt # Matplotlib for displaying images

# Load YOLOv8-small model
model = YOLO("yolov8s.pt") # Load the pre-trained YOLOv8 model

# Configure depth and color streams for RealSense camera
pipeline = rs.pipeline() # Create a pipeline to manage camera streaming
config = rs.config()

# Enable the color stream (640x480 resolution, 30 FPS)
config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)

# Start streaming from the RealSense camera
pipeline.start(config)

# Create a figure for real-time display using Matplotlib
fig, ax = plt.subplots()

try:
    while True:
        # Wait for a coherent pair of frames: color frame
        frames = pipeline.wait_for_frames()
        color_frame = frames.get_color_frame()

        if not color_frame:
            continue # Skip loop iteration if no color frame is
available

        # Convert the RealSense color frame to a numpy array
        color_image = np.asanyarray(color_frame.get_data())

        # Run YOLOv8 inference on the captured frame
        results = model(color_image) # Perform object detection

        # Loop through results and annotate the frame
        for result in results:
            annotated_img = result.plot() # Get an annotated image with
detections
```



```

# Convert BGR to RGB for correct colors in Matplotlib
annotated_img_rgb = annotated_img[..., ::-1]

# Display the resulting frame using Matplotlib
ax.imshow(annotated_img_rgb)
ax.axis("off") # Remove axes for better visualization
plt.draw() # Update the plot in real-time
plt.pause(0.001) # Short pause to allow real-time display
ax.clear() # Clear the axis for the next frame

finally:
    # Stop streaming from the RealSense camera
    pipeline.stop()
    plt.close(fig) # Close the Matplotlib window

```

Step 6: Final Setup And Integration of Trained Model(best.pt) + Real Sense + VS Code

"Download the best.pt file from Google Colab after training the model and save it in the same directory as your project folder."

Here I am providing you the code :

```

import cv2
import numpy as np
import pyrealsense2 as rs
import torch
from ultralytics import YOLO
# Check CUDA availability
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
# Load YOLOv8 model on CUDA
model = YOLO("best.pt").to(device) #best.pt has same directory as
your.py file
# Configure RealSense camera
pipeline = rs.pipeline()
config = rs.config()
config.enable_stream(rs.stream.color, 1280, 720, rs.format.bgr8, 30)
# Start RealSense camera
pipeline.start(config)

```

```

try:
    while True:
        # Get RealSense frames
        frames = pipeline.wait_for_frames()
        color_frame = frames.get_color_frame()
        if not color_frame:
            continue
        # Convert frame to NumPy array
        frame = np.asanyarray(color_frame.get_data())
        # Perform YOLO detection on GPU
        results = model(frame, device=device)
        # Draw detection results on the frame
        frame_with_detections = results[0].plot()
        # Display the frame
        cv2.imshow('Basketball Hoop Detection', frame_with_detections)
        # Press 'q' to exit
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

finally:
    pipeline.stop()
    cv2.destroyAllWindows()

```

From this code you will also know your device running on GPU or CPU.

Congratulations : Your setup is done.

